

Enrique Baquela - Andrés Redchuk

**Optimización
Matemática con R.
Volumen I:
Introducción
al modelado
y resolución
de problemas**

Optimización Matemática con R

Volumen I: Introducción al modelado y resolución de problemas

Enrique Gabriel Baquela

Andrés Redchuk

Madrid, Marzo de 2013

© Optimización matemática con R. Volumen I
Introducción al modelado y resolución de problemas
© Enrique Gabriel Baquela
© Andrés Redchuk
1ª edición, Mayo de 2013
ISBN: 978-84-686-3748-8
Editor Bubok Publishing S.L.
Impreso en España / Printed in Spain

Dedicatorias

*A Clementina Augusta, por existir.
A Flavia Sabrina, por ayudarme a ser lo que soy.
A mis padres y hermanos, por estar siempre.
A Isidoro Marín, Horacio Rojo,
Silvia Ramos y Gloria Trovato,
por el empujón inicial.
A mi directora de Doctorado, Ana Carolina Olivera,
y a mi codirectora, Marta Liliana Cerrano
por aceptar el desafío de ayudarme
a formarme en investigación.
A mis amigos y compañeros del GISOL,
por apoyarme desde el principio.
A Juan Cardinalli, por sus apreciaciones
diarias dentro del trabajo de Furgens Research.
A Andrés, por animarme y ser mi socio
en esta aventura.*

Enrique Gabriel Baquela

*A las tres hijas que Rita me dio
Sophie, Vicky, Érika,
y a los tres ahijados que la vida regaló
Leo, Fede y Alejo.*

Andrés Redchuk

Autores

Enrique Gabriel Baquela

Enrique Gabriel Baquela es Ingeniero Industrial por la Universidad Tecnológica Nacional de San Nicolás de los Arroyos (UTN-FRSN), y alumno de Doctorado en la Universidad Nacional de Rosario (UNR).

Es coordinador del Grupo de Investigación en Simulación y Optimización Industrial (GISOI), investigador del departamento de Ingeniería Industrial de UTN-FRSN, profesor de Simulación de procesos y sistemas y de Optimización Aplicada en el mismo centro, y co-fundador de Furgens Research.

Andrés Redchuk

Andrés Redchuk es Ingeniero Industrial por la Universidad de Lomas de Zamora en Buenos Aires, Master en Calidad Total y en Ingeniería Matemática de la Universidad Carlos III de Madrid; Master en Tecnologías de la Información y Sistemas Informáticos y Doctor en Informática y Modelización Matemática de la Universidad Rey Juan Carlos en España.

Andrés Redchuk es Master Black Belt de la Asociación Española para la Calidad, profesor de la Facultad de Ciencias Empresariales de la Universidad Autónoma de Chile, Profesor de la Facultad de Ingeniería de la Universidad Nacional de Lomas de Zamora, miembro del departamento de Estadística e Investigación Operativa de la Universidad Rey Juan Carlos en Madrid, España y Director del Grupo de Investigación en Lean Six Sigma.

Prologo

Los problemas del mundo real son, en general, no lineales enteros mixtos y de gran dimensión. Poder resolver esa clase de problemas es de vital importancia para todo profesional. Encontrar la solución óptima a estos problemas suele ser nuestro principal objetivo, aunque muchas veces nos alcanza con encontrar soluciones con ciertas características, aunque se trate de un resultado subóptimo y conocer la “calidad” de esta solución.

Es conveniente tener disponibles algoritmos eficientes y robustos para la resolución es esa clase de problemas.

El software R, ha demostrado ser muy eficiente para programar e implementar las técnicas estadísticas y los algoritmos de Optimización más tradicionales.

Mediante este libro pretendemos divulgar el desarrollo y la programación de los algoritmos más tradicionales y conocidos de optimización. Además presentamos su implementación en R. Para encontrar soluciones óptimas desarrollamos algoritmos de optimización y para encontrar soluciones cuasi óptimas desarrollamos algoritmos metaheurísticos.

El libro está pensado para estudiantes de carreras universitarias y de cursos de posgrado que quieran trabajar en el campo de la Optimización, de la Programación Matemática, los Métodos Cuantitativos de Gestión y la Investigación Operativa. Presenta herramientas para utilizar a modo de caja negra, como también el diseño de algoritmos específicos.

Los autores

¿Cómo leer este libro?

Referencias

Este libro es principalmente de aplicación práctica. Gran parte de su contenido consiste en código para transcribir directo a la consola de R. Por lo tanto, hemos adoptado una convención respecto a la tipografía para que las partes de este libro sean fácilmente identificables:

Cada vez que el Texto tenga este formato, entonces es texto explicatorio y formará parte del hilo conductor del libro.

Texto con este formato, es código R, ya sea para ingresar a la consola o bien para grabar en un script. Los comentarios, que se indican con #, se pueden suprimir. R no termina una sentencia hasta que no cerremos sintácticamente a la misma, así que podemos escribirlas en varias líneas como a veces, por cuestión de espacio, aparece en este libro.

Texto con este formato se usa para mostrar la salida de consola. Es lo que nos debería devolver R tras ingresar los comandos indicados. No siempre se incluye la salida de consola, cuando se considera que su inclusión es redundante la omitimos en este libro.

TEXTO CON ESTE FORMATO ES UN FORMULA O MODELO MATEMÁTICO

Como comentario extra, los bloques de código no son independientes unos de otros. Al inicio de cada capítulo tenemos siempre el siguiente código:

```
rm(list=ls())  
setwd("D://Proyectos//IOR//scripts")
```

La primera línea borra la memoria, mientras que la segunda línea declara cual es el directorio de trabajo. Adicionalmente, podemos encontrar este código de borrado repartido por el código de un mismo capítulo.

```
rm(list=ls())
```

Cuando encontramos esta línea, borrando la memoria de trabajo, significa que todo el código posterior a esa sentencia es independiente del anterior, así que podemos trabajar tranquilos. Pero, si buscando por el libro encontramos un código que nos interesa sin la llamada a

rm(list=ls()), debemos buscar la llamada inmediata anterior y tipear todo el código que existe a partir de ella (incluyendo el *rm(list=ls())*).

Estructura del libro

Este volumen tiene dos partes bien diferenciadas, ocupando cada una un 50 por ciento del mismo. La primera, dedicada a la introducción al entorno y al lenguaje R, puede ser omitida por quien ya tenga conocimientos del mismo. Si no es el caso, aconsejamos su lectura y la ejecución de todo el código proporcionado, siguiendo el orden prefijado por el libro. R tiene una sintaxis y una filosofía propia y diferente a la que nos tienen acostumbrados los lenguajes similares a C/C++ y Pascal. Pensado para el uso en problemas matemáticos en general, y estadísticos en particular, tiene una expresividad enorme que ayuda a no alejarnos de los problemas que estamos intentando resolver. Particularmente importante son los conceptos de vectorización y reciclado, a los que el usuario tendrá que acostumbrarse para sacar el máximo provecho de esta herramienta.

Por otro lado, la segunda parte está enfocada a la implementación y ejecución de distintas técnicas algorítmicas para resolución de problemas de programación matemática. Cada capítulo es independiente de los demás, pudiéndose leer en cualquier orden. Sin embargo, sin importar el algoritmo que se requiera utilizar, aconsejamos la lectura del apartado “Optimizando en base a muestras” del capítulo “Algoritmos para programación Lineal”, ya que allí se muestra un puente entre el procesamiento estadístico de datos, el armado de modelo lineales y su posterior optimización (que, con un poco de esfuerzo, pueden ser migrados a otros algoritmos de optimización e incluso a problemas no lineales).

Índice de contenido

¿Cómo leer este libro?	vii
Referencias	vii
Estructura del libro	ix
Índice de contenido	xi
Introducción	13
1.1. ¿Qué es un problema de optimización?	13
1.2. Modelado de sistemas Vs Resolución de modelos	14
1.3. Clasificación de problemas de optimización	14
1.4. Clasificación de métodos de resolución	15
<i>Resolución mediante cálculo</i>	15
<i>Resolución mediante técnicas de búsquedas</i>	16
<i>Resolución mediante técnicas de convergencia de soluciones</i>	16
1.5. Optimización Convexa	17
Introducción al uso de R	19
2.1 Actividades previas.	19
2.2. Utilizando R	21
2.3. Acerca del trabajo con matrices en R	40
2.4. Arrays	48
2.5. Listas	50
Un poco más de R	53
3.1. Utilizando funciones en R	53
3.2. Guardando nuestras funciones	58
3.3. Armando scripts	59
Gráficos con R	61
4.1. Trabajando con el paquete graphics	61
4.2. Trabajando con layouts	75
Implementando nuestras primeras búsquedas	83
5.1. Acerca de las búsquedas por fuerza bruta	83
5.2. Búsqueda aleatoria	89
5.3. Búsqueda dirigida por gradientes	91
Algoritmos para programación lineal	93
6.1. Introducción a la Programación Lineal	93

6.2. Cómo resolver problemas de PL	93
6.3. Otros paquetes de PL: lpSolve	96
6.4. Otros paquetes de PL: lpSolveAPI	105
6.5. Optimizando en base a muestras	109
Algoritmos para programación no lineal	115
7.1. La función OPTIM	115
7.2. Optimizando con restricciones	123
7.3. Armando nuestra propia versión de OPTIM	124
Algoritmos genéticos	129
8.1. Algoritmos genéticos	129
8.2. Armando nuestras propias funciones	137
Anexo I: Interacción con RStudio	145
¿Por qué utilizar RStudio?	145
Configurando el layout de visualización	145
Trabajando con paquetes	146
Consultando la ayuda	147
Visualizando gráficos	147
Monitoreando la memoria de trabajo	148
Revisando el historial	149
Escribiendo nuestros script	149
Ejecutando View()	150
Anexo II: Más paquetes para optimización	151
Como buscar más paquetes de optimización	151
Bibliografía	153

Introducción

1.1. ¿Qué es un problema de optimización?

Dentro de la investigación operativa, las técnicas de optimización se enfocan en determinar la política a seguir para maximizar o minimizar la respuesta del sistema. Dicha respuesta, en general, es un indicador del tipo “Costo”, “Producción”, “Ganancia”, etc., la cual es una función de la política seleccionada. Dicha respuesta se denomina objetivo, y la función asociada se llama función objetivo.

Pero, ¿qué entendemos cómo política? Una política es un determinado conjunto de valores que toman los factores que podemos controlar a fin de regular el rendimiento del sistema. Es decir, son las variables independientes de la función que la respuesta del sistema.

Por ejemplo, supongamos que deseamos definir un mix de cantidad de operarios y horas trabajadas para producir lo máximo posible en una planta de trabajos por lotes. En este ejemplo, nuestro objetivo es la producción de la planta, y las variables de decisión son la cantidad de operarios y la cantidad de horas trabajadas. Otros factores afectan a la producción, como la productividad de los operarios, pero los mismos no son controlables por el tomado de decisiones. Este último tipo de factores se denominan parámetros.

En general, el quid de la cuestión en los problemas de optimización radica en que estamos limitados en nuestro poder de decisión. Por ejemplo, podemos tener un tope máximo a la cantidad de horas trabajadas, al igual que un rango de cantidad de operarios entre los que nos podemos manejar. Estas limitaciones, llamadas “restricciones”, reducen la cantidad de alternativas posibles, definiendo un espacio acotado de soluciones factibles (y complicando, de paso, la resolución del problema). Observen que acabamos de hablar de “soluciones factibles”. Y es que, en optimización, cualquier vector con componentes apareadas a las variables independientes que satisfaga las restricciones, es una solución factible, es decir, una política que es posible de implementar en el sistema. Dentro de las soluciones factibles, pueden existir una o más soluciones óptimas, es decir, aquellas que, además de cumplir con todas las restricciones, maximizan (o minimizan, según sea el problema a resolver) el valor de la función objetivo.

Resumiendo, un problema de optimización está compuesto de los siguientes elementos:

- Un conjunto de restricciones
- Un conjunto de soluciones factibles, el cual contiene todas las posibles combinaciones de valores de variables independientes que satisfacen las restricciones anteriores.
- Una función objetivo, que vincula las soluciones factibles con la performance del sistema.

1.2. Modelado de sistemas Vs Resolución de modelos

Dentro de la práctica de la optimización, existen dos ramas relacionadas pero diferenciadas entre sí. Por un lado, nos encontramos con el problema de cómo modelar adecuadamente el sistema bajo estudio, y por otro como resolver el modelo. En general, la rama matemática de la IO se ocupa de buscar mejores métodos de solución, mientras que la rama ingenieril analiza formas de modelar sistemas reales.

Un ingeniero, en su actividad normal, se ocupa en general de modelar los sistemas, seleccionar un método de resolución y dejar que la computadora haga el resto. Sin embargo, es conveniente conocer el funcionamiento de cada metodología, a fin de determinar cuál es la más adecuada para cada situación, y modelar el problema de una forma amigable a dicho método.

1.3. Clasificación de problemas de optimización

En base a su naturaleza, hay varias formas de clasificar un problema de optimización. Analizar en qué categoría entra es importante para definir el método de solución a utilizar, ya que no hay un método único para todos los posibles problemas.

Para comenzar, una primera distinción la podemos realizar en base a la continuidad o no de las variables de decisión. Se dice que estamos frente a un problema de "Optimización Continua" cuando todas las variables de decisión pueden tomar cualquier valor perteneciente al conjunto de los reales. Dentro de este tipo de problemas, son de particular importancia los problemas de "Optimización Convexa", en los cuales se debe minimizar una función convexa sujeta a un conjunto solución convexo.

Cuando tanto la función objetivo como las restricciones son lineales, hablamos de un problema de "Optimización Convexa Lineal" o "Programación Lineal". En el caso de trabajar con variables discretas (es decir, que solo puedan tomar valores enteros) nos enfrentamos a un problema de "Optimización Combinatoria". Por raro que pueda parecer, en general un problema de optimización combinatoria es más complicado de resolver que uno de optimización continua. En el medio tenemos los problemas de "Optimización Mixta" es los cuales algunas variables son continuas y otras son discretas. En la práctica, estos problemas se resuelven en forma más parecida a los problemas combinatorios que a los continuos. Un caso particular de optimización combinatoria es la "Optimización Binaria", aquella en la cual todas sus variables están restringidas a tomar uno de dos valores (en general, 0 y 1). Este caso es bastante raro de encontrar en la práctica, siendo más habitual encontrar problemas combinatorios con algunas variables binarias (optimización mixta).

Otra clasificación la podemos hacer en base a la naturaleza probabilística del problema. Cuando podemos considerar que todas las variables son determinísticas, estamos ante un problema determinista, en caso contrario nos enfrentamos a un problema estocástico. El modelado y resolución de un problema estocástico es mucho más complejo que el modelado de un problema determinístico.

1.4. Clasificación de métodos de resolución

Los métodos de resolución de problemas de optimización se pueden clasificar en tres tipos diferentes:

- Resolución mediante cálculo
- Resolución mediante técnicas de búsquedas
- Resolución mediante técnicas de convergencia de soluciones

Resolución mediante cálculo

Los métodos de resolución por cálculo apelan al cálculo de derivadas para determinar para qué valores del dominio la función presenta un máximo o un mínimo. Son métodos de optimización muy robustos, pero que requieren mucho esfuerzo de cómputo y que tanto la función objetivo como las restricciones presenten determinadas condiciones (por

ejemplo, que estén definidas, que sean continuas, etc.). En general, no se suele apelar a estos métodos en el mundo de la ingeniería, ya que los problemas no se ajustan a las restricciones de continuidad y tienen demasiadas variables como para que su tratamiento pueda ser eficiente.

Resolución mediante técnicas de búsquedas

Dentro de este apartado, podemos encontrar un gran abanico de técnicas, desde el viejo método de prueba y error hasta las modernas técnicas de programación matemática. En forma genérica, consisten en el siguiente algoritmo:

Seleccionar una solución inicial y hacerla la solución actual:

- 1) Hacer Mejor Solución = Solución Actual
- 2) Buscar n soluciones cercanas a la Solución Actual
- 3) Para cada una de las n soluciones cercanas hacer
 - a. Si el valor de la función objetivo de la solución a verificar es mayor (o menor) al valor generado por la solución actual, hacer Mejor Solución = Solución Evaluada
 - b. Si Mejor Solución = Solución Actual, Finalizar del procedimiento, en caso contrario Hacer Solución Actual = Mejor Solución y volver a 2)

Dentro de estos métodos tenemos técnicas para abarcar una gran variedad de problemas. Desde técnicas exactas, como la “Programación Lineal” (que se limita solo a problemas con un conjunto solución convexo y función objetivo y restricciones lineales) hasta las técnicas metaheurísticas de solución aproximada como la “Búsqueda Tabú”.

Resolución mediante técnicas de convergencia de soluciones

Dentro de este grupo, tenemos las técnicas más recientemente desarrolladas. A diferencia del conjunto anterior, está compuesto casi completamente por técnicas metaheurísticas (o sea, que los resultados van a ser aproximadamente óptimos). Estos métodos se basan en generar una gran cantidad de soluciones, determinar cuáles son las “mejores” y, a partir de ellas, generar un nuevo conjunto de soluciones a analizar, repitiendo el proceso hasta que las soluciones generadas

converjan en una (o sea, hasta la iteración en la cual todas las soluciones generadas tengan un valor de función objetivo muy parecido).

Entre las técnicas más conocidas de este grupo, aunque no las únicas, tenemos a todas las versiones de "Algoritmos Genéticos"

1.5. Optimización Convexa

La Optimización Convexa es una rama de las técnicas de Optimización que trata sobre técnicas de minimización de funciones convexas sobre un dominio también convexo.

Definamos primero que es una función convexa. Una función es convexa, o cóncava hacia arriba, si, para todo par de puntos pertenecientes a su dominio, la recta que los une pertenece o está sobre la curva. En otras palabras, una función es convexa si el conjunto de puntos situados sobre su gráfica es un conjunto convexo. Matemáticamente, una función cumple con estas características si, para todo X e Y perteneciente a su dominio, con $X < Y$, y un todo T tal que $0 < T < 1$, se verifica:

$$f(tx + (1-t)y) \leq t f(x) + (1-t) f(y)$$

De más está decir que la función debe estar definida para todos los reales, al igual que su codominio.

Un conjunto de puntos de un espacio vectorial real se denomina conjunto convexo si, para cada par de puntos pertenecientes al conjunto, la recta que los une también pertenece al conjunto. Matemáticamente:

Dado el conjunto C , si para todo $(A;B)$ perteneciente a C y todo T perteneciente a $[0;1]$, se cumple que $(1-T)A + TB$ pertenece a C .

Los problemas de optimización convexa toman la forma de una función objetivo convexa a minimizar y un conjunto de restricciones que cumplen las siguientes condiciones:

- Si son inecuaciones, son de la forma $g(x) \leq 0$, siendo g una función convexa.
- Si son ecuaciones, son de la forma $h(x) = 0$, siendo h una función afín.

En realidad, las ecuaciones son redundantes, ya que se pueden reemplazar por un par de inecuaciones de la forma $h_i(x) \leq 0$ y $-h_i(x) \leq 0$.

Noten que si la función $h(x)$ no es afín, $-h(x)$ es cóncava, por lo cual el problema dejaría de ser convexo.

Cumpliendo las condiciones anteriores, nos aseguramos que el problema se puede resolver por los métodos de optimización convexa.

Es fácil darse cuenta que, en el caso de necesitar maximizar una función cóncava, es fácil transformar el problema en uno de minimización de función convexa, haciendo $f_t = -f$. También, si las inecuaciones son de la forma $h(x) \geq 0$ podemos transformarlas a $h_t(x) \leq 0$ mediante la transformación $h_t = -h$.

Dentro de los métodos de resolución de estos tipos de problemas podemos nombrar los siguientes:

- Método Simplex (para problemas convexos lineales)
- Métodos de punto interior
- Resolución por elipsoides
- Resolución por subgradientes
- Resolución por proyección de subgradientes

En general, la mayoría de los problemas de optimización con variables reales suelen entrar dentro de esta categoría, por lo cual con el aprendizaje de este tipo de técnicas tenemos gran parte del camino allanado.

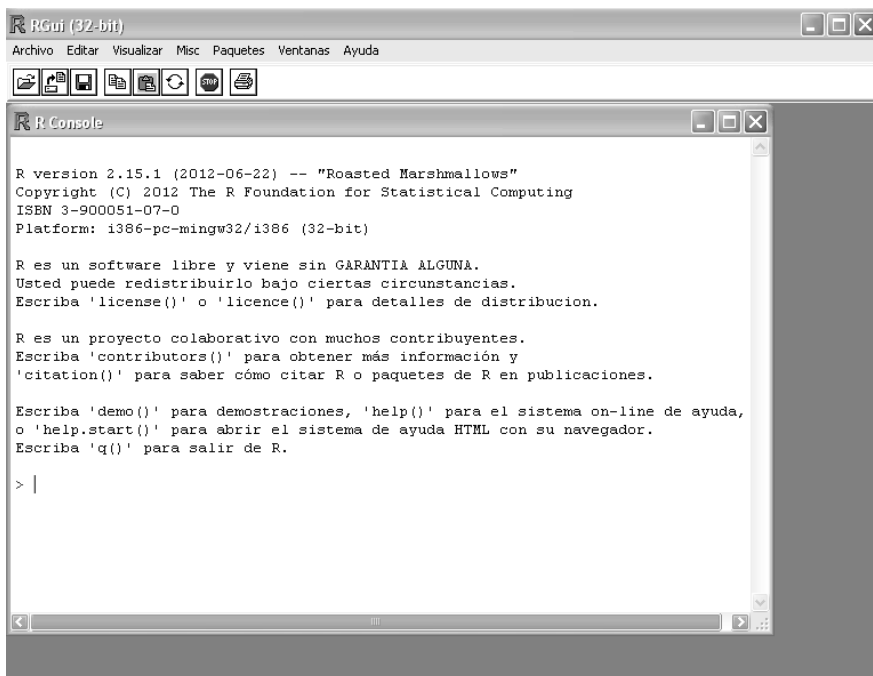
Dentro de los casos especiales, tenemos los siguientes tipos de problemas:

- Programación Lineal: Optimización convexa con función objetivo y restricciones lineales.
- Programación Cuadrática Convexa: Optimización convexa con restricciones lineales pero función objetivo cuadrática.
- Programación No Lineal Convexa: Optimización convexa con la función objetivo y/o las restricciones no lineales.

Introducción al uso de R

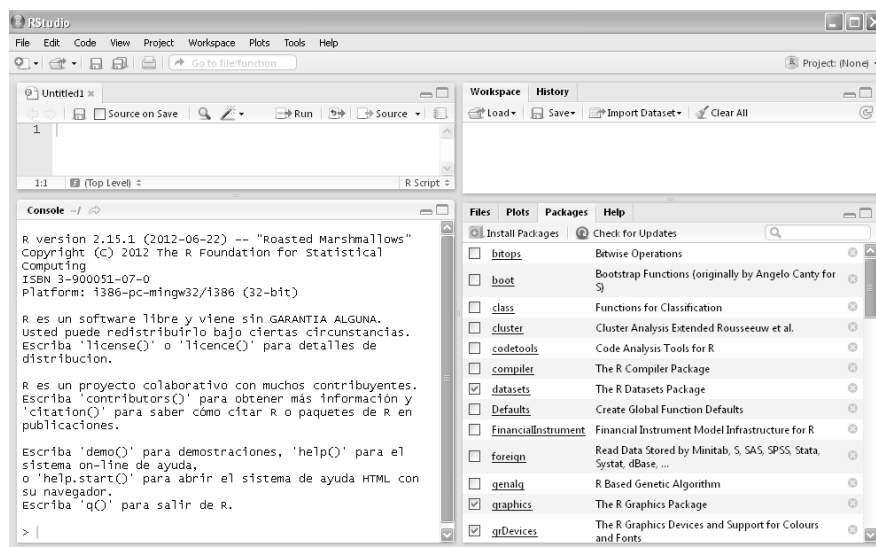
2.1 Actividades previas.

Antes de comenzar a trabajar, tenemos que instalarnos el software R. El mismo se puede descargar desde la página de CRAN (<http://cran.r-project.org/>). Una vez instalado, su aspecto es el siguiente:



La interfaz de R es una consola de comandos convencional, no viene con ningún IDE más avanzado. Pero podemos utilizar algunas de las muchas IDEs preparadas para R. En este libro vamos a trabajar con RStudio, pero no es de ninguna manera necesario tenerlo instalado. Todo R se puede manejar con comandos en la consola, así que podemos usar la consola básica, RStudio, o cualquier otra IDE sin perder el hilo del libro. De hecho, solo en algunas gráficas de este libro (principalmente algún que otro cuadro de diálogo en la ejecución del comando `View()`) se apreciará la

interfaz de RStudio. En el Anexo I hay un pequeño tutorial mostrando las características de esta IDE.



Amen a tener instalado el sistema, es necesario crear una estructura de directorios para trabajar con R. Creemos una estructura como la que sigue:

Directorio de trabajo

|__scripts

|__sources

|__data

Como dijimos antes, la interacción con R será mediante sentencias. Así que debemos ingresar el texto escrito en formato de código y pulsar enter para ejecutarlo. Si lo escribimos en forma incompleta (por ejemplo, nos olvidamos de cerrar un paréntesis), R se queda esperando a que terminemos la sentencia, tras lo cual debemos volver a pulsar enter. Si la construcción es sintácticamente incorrecta, la consola devuelve error.

2.2. Utilizando R

No hay mejor forma que aprender un nuevo lenguaje de programación y un nuevo software que usándolo. Así que vamos a empezar a trabajar con R. Suponemos que ya lo tenemos instalado, así que abrimos la consola de R o el IDE que estamos utilizando. Primero, limpiemos la memoria y definamos nuestro directorio de trabajo:

```
rm(list=ls())  
setwd("D://Proyectos//IOR//scripts")
```

Donde debemos reemplazar "D://Proyectos//IOR//" por la ruta hacia el directorio para alojar nuestros proyectos en R. Referenciamos al subdirectorio script ya que en él es donde guardaremos el código que generemos en R.

Expliquemos qué hemos escrito. La segunda línea es fácil, "setwd()" es la abreviatura de "Set Working Directory" y lo que hace es definir a un directorio como directorio de trabajo (o sea, que cada vez que guardemos o leamos un archivos, a menos que escribamos una ruta, las operaciones se hacen en dicho directorio).

Si usamos *getwd()* ("Get Working Directory"):

```
getwd()  
[1] "D://Proyectos//IOR//scripts"
```

Obtenemos la ruta del directorio de trabajo. Con *dir()* podemos ver su contenido:

```
dir()
```

Volvamos al comando "rm(list=ls())". El mismo se compone de una función anidada dentro de otra. *ls()* devuelve una lista con el contenido de la memoria de trabajo. *rm()* borra de la memoria los objetos que le indiquemos. Entonces *rm(list=ls())* borra todos los objetos que se encuentren en la memoria de trabajo.

Por ejemplo, creemos algunos objetos:

```
aux01 <- 1  
aux02 <- 2  
aux03 <- 3
```

Si quiero ver los objetos en memoria:

```
ls()  
[1] "aux01" "aux02" "aux03"
```

Si quiero borrar "aux02":

```
rm(aux02)
```

Vuelvo a ejecutar *ls()* para ver si se borró:

```
ls()  
[1] "aux01" "aux03"
```

Borremos nuevamente todos los objetos

```
rm(list=ls())
```

Y verifiquemos si están borrados:

```
ls()  
character(0)
```

Los comentarios se declaran mediante una almohadilla "#"

```
# Esto es un comentario
```

Los objetos se crean mediante asignación de valores. El operador de asignación es "<-"¹:

```
aux01 <- "Variable Auxiliar 01"
```

Y podemos consultar su valor simplemente llamándola:

```
aux01  
[1] "Variable Auxiliar 01"
```

En general, si la variable contiene datos y no sabemos qué cantidad, debemos usar *View()* (ojo, R es sensible a mayúsculas y minúsculas):

```
View(aux01)
```

	x
1	Variable Auxiliar 01

Fíjense que pasa si hacemos *view(aux01)*:

¹ En realidad, también se puede emplear el operador "=", pero está desaconsejado. Las buenas prácticas dejan el "<-" como operador de asignación para variables, y limitan el "=" a la asignación de valores a parámetros de funciones. En este libro se seguirá dicho criterio.


```
view(aux01)
Error: no se pudo encontrar la función "view"
```

Nos tira error, porque para R “*view*” es diferente a “*View()*” (R es case sensitive²). Este comando nos ayuda bastante porque, de tener una variable con muchos datos, el mostrarlos invocándola directamente nos desplazaría la consola, mientras que con “*View()*” los datos se muestran en una nueva ventana.

R viene con su propia ayuda incluida, a la que se accede de múltiples maneras.

Si queremos ver la ayuda de la función *View*:

```
help(View)
```

O bien:

```
?View()
```

Si queremos ver el manual de ayuda:

```
help.start()
```

Podemos buscar alguna palabra con *help.search()*:

```
help.search("normal")
```

Buscar algún comando por parte de su nombre con *apropos()*:³

```
apropos("chi")
```

También, si los paquetes⁴ lo permiten, podemos ver demostraciones de su uso mediante la función *demo()*. Por ejemplo, veamos la demostración del paquete “*graphics*”:

```
demo(graphics)
```

² Diferencia mayúsculas de minúsculas cuando le damos instrucciones.

³ OK, hubiera quedado mejor algo como “*help.apropos*”, pero es lo que hay.

⁴ Paquetes es el nombre que se le da en R a las bibliotecas empaquetadas. No son simplemente módulos que se cargan, sino que vienen con su propia ayuda, se instalan en el path de R, entre otros.

R es un lenguaje orientado a objetos. Cada elemento en memoria es un objeto, y si queremos acceder a alguno hay que asignarlo a alguna variable:

```
myVar01 <- 15
myVar01
[1] 15
```

Los objetos residen en memoria, las variables son meras referencias al objeto. Recordemos esto, la variable no es el objeto.

Cuando decimos que cada elemento en memoria es un objeto, nos referimos a todos: datos, funciones, gráficos, etc. Todo puede ser manipulado mediante variables. Cada objeto, indefectiblemente, tiene que pertenecer a una clase. R es un lenguaje de tipado dinámico y fuerte. O sea, puedo definir un objeto asignándole valores, que R se encarga de asignarle el tipo más apropiado (según su forma de ver las cosas), pero el mismo se mantiene inmutable el resto de la vida de dicho objeto.

De todas las clases disponibles, con las que más vamos a tratar son con las clases de datos. Los cuatro tipos básicos de datos son: "scalar", "factor", "character" y "logical".

"logical" es un tipo de dato simple, que permite guardar el valor de un bit. Es el clásico booleano de otros lenguajes de programación. Borremos la memoria de trabajo y pongámonos a trabajar con este tipo de datos:

```
rm(list=ls())

varBoolean01 <- TRUE
varBoolean02 <- FALSE
varBoolean03 <- TRUE

varBoolean01
[1] TRUE

varBoolean02
[1] FALSE

varBoolean03
[1] TRUE
```

Con `class()` chequeamos la clase a la que corresponde el objeto referenciado por la variable:

```
class(varBoolean01)
[1] "logical"
```

Obviamente, podemos realizar operaciones booleanas:

```
!varBoolean01                # Negación
[1] FALSE
```

```
varBoolean01 && varBoolean02      # Operación AND
[1] FALSE

varBoolean01 || varBoolean02      # Operación OR
[1] TRUE
```

Podemos hacer operaciones más complejas también:

```
!((varBoolean01 || varBoolean02) && (varBoolean03))
```

Podemos utilizar números mediante la clase "scalar". En realidad nunca tratamos con esta clase directamente, sino que operamos con sus clases hijas, "numeric" y "complex":

```
varNumeric01 <- 135
varNumeric02 <- -20574
varNumeric03 <- 12/27
varNumeric04 <- pi * 2
varNumeric05 <- 3.4

varComplex01 <- 12 + 2i
varComplex02 <- 134/245 - 3.1i
varComplex03 <- 946/24i
varComplex04 <- 123.45 + (2 * pi) * 1i

class(varNumeric01)
[1] "numeric"

class(varComplex04)
[1] "complex"
```

Puedo realizar todas las operaciones que uno espera poder hacer con números.⁵

```
varNumeric06 <- varNumeric01 + varNumeric03
varNumeric06

varNumeric01 + varNumeric03
varNumeric01 - varNumeric03
varNumeric01 * varNumeric03
varNumeric01 / varNumeric03 # División real
varNumeric02 ^ 2

varNumericInteger01 <- 10
varNumericInteger02 <- 7

varNumericInteger01 %% varNumericInteger02      # Módulo
```

⁵ Bueno, es bastante obvio, ¿no? Pero había que decirlo.

```
varNumericInteger01 %/% varNumericInteger02 # División
entera
```

Observen un detalle. Si realizo una operación y la asigno a una variable, su resultado es un objeto referenciado por esa variable, pero la consola no lo muestra, debo llamar al objeto para ver el resultado. Pero si realizo el cálculo sin realizar ninguna asignación, se crea el objeto, se muestra su valor en la consola (el resultado del cálculo) pero es borrado inmediatamente de la memoria de trabajo.

Volviendo al tema de los cálculos, también es posible realizar comparaciones, las cuales nos devuelven un objeto del tipo "logical":⁶

```
varNumeric01 > varNumeric03
auxVar01 <- varNumeric01 > varNumeric03
class(auxVar01)

varNumeric01 > varNumeric03
varNumeric01 >= varNumeric03
varNumeric01 == varNumeric03
varNumeric01 < varNumeric03
varNumeric01 <= varNumeric03
```

Y como las comparaciones son valores "logical", podemos realizar operaciones binarias:

```
(varNumeric01 >= varNumeric03) || (varNumeric01 <=
varNumeric02)
(varNumeric01 >= varNumeric03) && (varNumeric01 <=
varNumeric02)
!(varNumeric01 > varNumeric03)
```

"character" es un tipo de datos que nos permite almacenar cadenas de texto:

```
myCharacter01 <- "Mi primer cadena de texto"
myCharacter02 <- "b"
myCharacter03 <- "1222"

class(myCharacter01)
[1] "character"
```

Tenemos algunas funciones especiales para trabajar con cadenas de texto:

⁶ Debido a que el signo "=" se usa para asignar valores a parámetros de funciones, el operador para chequear igualdad es "==".

```
paste(myCharacter03,myCharacter02) # Concatenación
[1] "1222 b"

substr(myCharacter01,1,10) # Recorte
[1] "Mi primer "

sub("cadena de texto", "String",myCharacter01) # Sustitución
[1] "Mi primer String"
```

Para cualquiera de estas clases, tenemos funciones *is.type()* y *as.type()*⁷:

```
is.numeric(myCharacter01)
is.numeric(varNumeric01)

varNumeric01 + myCharacter03 # Esto genera un error
varNumeric01 + as.numeric(myCharacter03)
```

is.numeric(), por ejemplo, devuelve TRUE si la variable es numérica y FALSE en caso contrario, mientras que *as.numeric()* nos devuelve el contenido de la variable pasada como parámetro en formato numérico (si la conversión es posible, obviamente).

El último tipo de datos es el "factor", utilizado para representar variables categóricas.⁸

```
varPosiblesValores <- c("Alto", "Medio", "Bajo")
varFactor01 <- factor("Alto", levels=varPosiblesValores)
varFactor01
[1] Alto
Levels: Alto Medio Bajo

varFactor02 <- factor(c("Alto", "Alto", "Bajo", "Medio"),
levels=varPosiblesValores)
varFactor02
[1] Alto Alto Bajo Medio
Levels: Alto Medio Bajo
```

Los factores se construyen mediante la función *factor()* ya que es necesario, además del valor que tomé el factor (el primer parámetro), definirle los posibles valores que podía tomar (el parámetro "levels").

Los factores tienen asociadas algunas funciones interesantes:

⁷ Donde type se refiere al nombre de alguna de estas clases.

⁸ En este ejemplo introducimos el comando "c", abreviatura de concatenate. Es uno de los más usados en R, y permite unir varios datos en una única variable vectorial. A cada dato se puede acceder luego por subíndices.

```
levels(varFactor02)
[1] "Alto" "Medio" "Bajo"
```

```
table(varFactor02)
varFactor02
  Alto Medio Bajo
    2     1    1
```

Y aunque no tiene *as.factor* (en vez de eso se usa *factor*), si tiene *is.factor*:

```
is.factor(varFactor02)
```

Los tipos de datos anteriores (y cualquier objeto) puede tomar el valor NA. Este valor indica “no asignación” y se caracteriza por ser indefinido (es decir, no vale ni cero, ni falso, ni "", es indeterminado).

No es un tipo de dato, pero funciona parecido en algunas cosas:

```
aux01 <- NA
aux01
is.na(aux01)
```

Si intentamos averiguar su clase, devuelve "logical", sin importar que pueda contener la variable:

```
aux01<-"prueba"
class(aux01)
aux01<-NA
class(aux01)
```

Similar al NA, existe el valor NaN (Not a Number), que es el devuelto por algunas operaciones que no tengan sentido:

```
0/0
Inf/Inf
```

Como NA tiene su *is.na()*, NaN tiene su *is.nan()*:

```
is.nan(0/0)
is.nan(1/0)
```

Vimos antes que usamos el valor “Inf”. “Inf” y “-Inf” son otros dos valores especiales que identifican al infinito positivo y negativo respectivamente:

```
1/0
[1] Inf

-1/0
[1] -Inf
```

Existen las funciones *is.finite()* e *is.infinite()* asociadas a estos valores:

```
is.finite(1/0)
is.finite(1/Inf)
is.infinite(2^Inf)
```

Ahora bien, estos tipos de datos, así como están, la verdad que sirven muy poco. En el uso diario, los empleamos como ladrillos para estructuras más complejas. De los datos complejos, la base de todo R es el vector. Un vector se crea con la función *c()* (abreviatura de concatenate)

```
myNumericVector <- c(1,345.6, pi,1/3)
myNumericVector
[1] 1.0000000 345.6000000 3.1415927 0.3333333
```

Un vector es un conjunto de objetos ordenados del mismo tipo:

```
class(myNumericVector)
[1] "numeric"
```

Un vector puede tener cualquier cantidad de elementos, incluso solo uno, pero todos deben ser del mismo tipo básico. Se accede a los mismos mediante sub-índices:

```
myNumericVector[3]
[1] 3.141593
```

Cuidado, R comienza a indexar a partir de 1, no de 0⁹. Puedo ampliar un vector a discreción con la función *c()*:

```
myNumericVector <- c(myNumericVector, 123, 2*pi/3, NA, 10, -
23.9, NA, -14/13)
myNumericVector
[1] 1.0000000 345.6000000 3.1415927 0.3333333 123.0000000
2.0943951
[7] NA 10.0000000 -23.9000000 NA -1.0769231
```

Y acceder a valores por rango de subíndices:

```
myNumericVector[2:5]
[1] 345.6000000 3.1415927 0.3333333 123.0000000

myNumericVector[-1]
```

⁹ Pese a que esto nos encuentra desacostumbrados, es una muy buena idea que los índices comiencen por “1”. Por ejemplo, si calculamos la cantidad de elementos de un vector, el índice del último elemento del vector se corresponde con este número.

```
[1] 345.6000000 3.1415927 0.3333333 123.0000000 2.0943951
NA
[7] 10.0000000 -23.9000000 NA -1.0769231

myNumericVector[-1:-2]
[1] 3.1415927 0.3333333 123.0000000 2.0943951 NA
10.0000000
[7] -23.9000000 NA -1.0769231
```

Tenemos algunas funciones que se aplican a vectores. Por ejemplo, *length()* nos devuelve el tamaño del vector, y *order()* los índices del vector ordenados crecientemente según los valores de cada componente:

```
length(myNumericVector)
[1] 11

order(myNumericVector)
[1] 9 11 4 1 6 3 8 5 2 7 10
```

Usando la función *order()*, podemos devolver los elementos de un vector ordenados:

```
myNumericVector[order(myNumericVector)]
[1] -23.9000000 -1.0769231 0.3333333 1.0000000 2.0943951
3.1415927
[7] 10.0000000 123.0000000 345.6000000 NA NA
```

Los vectores en R tienen una peculiaridad muy especial y es que muchas de sus operaciones están vectorizadas. La vectorización es un mecanismo que permite aplicar una misma función o expresión a todos los elementos de un vector, sin necesidad de construir un procedimiento para iterar. Por ejemplo, si quiero obtener todos los valores no negativos del vector

```
myNumericVector>=0
[1] TRUE TRUE TRUE TRUE TRUE TRUE NA TRUE FALSE NA
FALSE

which(myNumericVector>=0)
[1] 1 2 3 4 5 6 8

myNumericVector[which(myNumericVector>=0)]
[1] 1.0000000 345.6000000 3.1415927 0.3333333 123.0000000
2.0943951
[7] 10.0000000
```

```
myNumericVector[myNumericVector>=0]
[1] 1.0000000 345.6000000 3.1415927 0.3333333 123.0000000
2.0943951
[7] NA 10.0000000 NA
```


Observen que si filtro sin usar el *which()*, me devuelve también los valores que no puede evaluar (los NA en este caso).

Si quiero devolver todos los valores que no sean NA:

```
myNumericVector[which(!is.na(myNumericVector))]  
[1] 1.0000000 345.6000000 3.1415927 0.3333333 123.0000000  
2.0943951  
[7] 10.0000000 -23.9000000 -1.0769231
```

Lo más interesante, es que podemos realizar asignaciones a los elementos del vector que cumplen con la condición dada en el filtro:

```
myNumericVector02 <- myNumericVector  
myNumericVector02[which(myNumericVector02>10)] <- 0  
myNumericVector02  
[1] 1.0000000 0.0000000 3.1415927 0.3333333 0.0000000  
2.0943951  
[7] NA 10.0000000 -23.9000000 NA -1.0769231
```

```
myNumericVector03 <-  
(myNumericVector02+5)[(myNumericVector02 +5) >2]  
myNumericVector03  
[1] 6.0000000 5.0000000 8.141593 5.333333 5.000000 7.094395  
NA  
[8] 15.000000 NA 3.923077
```

Asociado al concepto de vectorización está el de reciclado (recycling en inglés), gracias al cual podemos hacer cosas como:

```
myNumericVector/10  
[1] 0.10000000 34.56000000 0.31415927 0.03333333 12.30000000  
0.20943951  
[7] NA 1.00000000 -2.39000000 NA -0.10769231  
  
myNumericVector - 20  
[1] -19.000000 325.600000 -16.85841 -19.66667 103.00000 -17.90560  
NA  
[8] -10.00000 -43.90000 NA -21.07692
```

Esto funciona porque R transforma a 10 y 20 en vectores de igual longitud que myNumericVector:

```
myNumericVector/c(10,10,10,10,10,10,10,10,10,10,10)  
myNumericVector04 <-  
myNumericVector/c(10,10,10,10,10,10,10,10,10,10,10)
```

Obviamente, esto funciona con vectores numéricos, no tiene sentido en vectores de caracteres.

Puedo generar secuencias y grabarlas en mi vector:

```
myNumericVector04 <- seq(1:10)
```

```

myNumericVector04
[1] 1 2 3 4 5 6 7 8 9 10

myNumericVector04 <- seq (from= 1, to= 10, by=2)
myNumericVector04
[1] 1 3 5 7 9

myNumericVector04 <- rep(1:4, 2)
myNumericVector04
[1] 1 2 3 4 1 2 3 4

myNumericVector04 <- rep(1:4, each=2)
myNumericVector04
[1] 1 1 2 2 3 3 4 4

myNumericVector04 <- rep(1:4, each=2, len=5)
myNumericVector04
[1] 1 1 2 2 3

myNumericVector04 <- rep(1:4, each=2, times=3)
myNumericVector04
[1] 1 1 2 2 3 3 4 4 1 1 2 2 3 3 4 4 1 1 2 2 3 3 4 4

myNumericVector04 <- rep(myNumericVector04, times=2)
myNumericVector04
[1] 1 1 2 2 3 3 4 4 1 1 2 2 3 3 4 4 1 1 2 2 3 3 4 4
[33] 1 1 2 2 3 3 4 4 1 1 2 2 3 3 4 4

```

Ahora bien, los vectores son un tipo de dato interesante, pero son unidimensionales. ¿Qué pasa si quiero manejar datos de altura discriminados por sexo?:¹⁰

```

altura <- rnorm(10, 1.70, 0.1)
altura
[1] 1.651319 1.779054 1.692859 1.637679 1.669193 1.699384 1.810102
[8] 1.822464 1.911979 1.685002

sexo <- sample(factor(c("M","F")), size=10, replace=TRUE)
sexo
[1] M M F F F F M M F F
Levels: F M

```

Si me aseguro que los subíndices de cada vector sean equivalentes, para saber el sexo y la altura del tercer elemento muestral:

```

sexo[3]
[1] F
Levels: F M

```

¹⁰ Para generar los valores utilicé la función *rnorm*, que genera valores aleatorios con distribución normal, y *sample*, que sirve para realizar muestreos de variables discretas.

```
altura[3]
[1] 1.692859
```

Puede ser algo incómodo, ¿no?¹¹ Probemos usar un "data.frame":

```
datosPoblacion <- data.frame(sexo, altura)
datosPoblacion
  sexo  altura
1    M 1.651319
2    M 1.779054
3    F 1.692859
4    F 1.637679
5    F 1.669193
6    F 1.699384
7    M 1.810102
8    M 1.822464
9    F 1.911979
10   F 1.685002
```

Mucho mejor, ¿no? Accedamos al tercer elemento:

```
datosPoblacion[3,]
  sexo  altura
3    F 1.692859
```

O mostremos solo la altura del tercer elemento:

```
datosPoblacion[3,2]
[1] 1.692859
```

También puedo mostrar todas las alturas:

```
datosPoblacion[,2]
[1] 1.651319 1.779054 1.692859 1.637679 1.669193 1.699384 1.810102
[8] 1.822464 1.911979 1.685002
```

Si quiero, puedo ver las alturas "con título":

```
datosPoblacion[2]
  altura
1 1.651319
2 1.779054
3 1.692859
4 1.637679
5 1.669193
6 1.699384
7 1.810102
8 1.822464
9 1.911979
10 1.685002
```

Cuando el data.frame es grande, conviene usar *View()*:

¹¹ Y sobre todo propenso a errores.

```
View(datosPoblacion)
```

	sexo	altura
1	M	1.651319
2	M	1.779054
3	F	1.692859
4	F	1.637679
5	F	1.669193
6	F	1.699384
7	M	1.810102
8	M	1.822464
9	F	1.911979
10	F	1.685002

También puedo acceder a las columnas por nombre utilizando el signo “\$”:

```
datosPoblacion$altura  
[1] 1.651319 1.779054 1.692859 1.637679 1.669193 1.699384 1.810102  
[8] 1.822464 1.911979 1.685002
```

```
datosPoblacion$altura[3]  
[1] 1.692859
```

Cada columna, y cada fila, son vectores hechos y derechos, así que puedo aplicarles filtros:

```
datosPoblacion[datosPoblacion$sexo == "M",]  
  sexo  altura  
1    M 1.651319  
2    M 1.779054  
7    M 1.810102  
8    M 1.822464
```

Observen la coma al final del filtro. Le indica a R que no queremos filtrar por columnas (en realidad, les estamos pasando un filtro nulo). Puedo ampliar un data.frame cuando quiera, por ejemplo, ahora vamos a agregarle un campo “peso”:

```
datosPoblacion$peso <- rnorm(10, 80, 15)  
View(datosPoblacion)
```

	sexo	altura	peso
1	F	1.676572	65.17624
2	M	1.897857	84.37898
3	F	1.771568	93.21675
4	M	1.728978	76.59352
5	F	1.675330	77.48651
6	F	1.888984	95.62467
7	M	1.766966	90.38411
8	F	1.710781	79.98631
9	F	1.796912	70.57039
10	M	1.681548	45.71884

Ni siquiera hubo que declararlo, solo poner datos. También funciona:¹²

```
datosPoblacion <- cbind(datosPoblacion, rnorm(10, 50, 15))
View(datosPoblacion)
```

	sexo	altura	peso	rnorm(10, 50, 15)
1	F	1.676572	65.17624	43.19132
2	M	1.897857	84.37898	38.27734
3	F	1.771568	93.21675	67.61725
4	M	1.728978	76.59352	38.63001
5	F	1.675330	77.48651	21.53811
6	F	1.888984	95.62467	69.04719
7	M	1.766966	90.38411	53.62271
8	F	1.710781	79.98631	19.74455
9	F	1.796912	70.57039	38.69197
10	M	1.681548	45.71884	46.87654

Qué problema, no tenemos nombre para la nueva columna:

```
names(datosPoblacion)
[1] "sexo" "altura" "peso"
[4] "rnorm(10, 50, 15)"

names(datosPoblacion) <-c(names(datosPoblacion[1:3]), "edad")
names(datosPoblacion)
[1] "sexo" "altura" "peso" "edad"
```

¹² *cbind* es equivalente a la función *c* pero para data frames. *cbind* concatena columnas a nuestro data frame. También existe *rbind*, que concatena filas a nuestro data frame.